Network-Based Phishing Detection Tool

Mini Project – I

B.Tech in Computer Science & Engineering (Cyber Security)

Academic Year 2025–2026

Team Details: Ankit Saraswat (Team Leader)-Project manager/coordinator

Nancy Paul-Dataset & UI/UX & Dashboard designer

Sujal Singh-Documentation lead and reasearch

Riya Devi- Presentation and Visualization

Department of Computer Science

GLA University, Mathura

The Phishing Threat Landscape

Phishing remains one of the most pervasive and dangerous cyber attacks, targeting users' sensitive information including passwords, banking credentials, and personal data.

Attackers craft convincing replicas of legitimate websites, exploiting human trust and making detection challenging for both users and traditional security systems. The sophistication and frequency of these attacks continue to escalate alongside growing internet adoption.

Our project addresses this critical security gap through real-time network traffic analysis.

Why Network-Based Detection Matters

Short-Lived Threats

Phishing websites operate for mere hours, rendering blacklist-based detection obsolete before updates can propagate.

Zero-Day Vulnerabilities

Traditional systems fail against newly created domains that haven't yet been catalogued or reported.

Real-Time Protection

Network-level detection identifies threats before users engage, providing a critical first line of defence.

Educational Relevance

Enhances cybersecurity awareness whilst providing practical experience in network security implementation.

Problem Statement

The Challenge

Phishing websites successfully impersonate legitimate platforms to harvest user credentials and sensitive data through social engineering.

Current Limitations

- Heavy reliance on pre-compiled blacklists that lag behind emerging threats
- Signature-based detection fails against novel attack patterns
- Zero-day phishing domains bypass traditional security measures
- No dynamic analysis of DNS and HTTP traffic patterns

Solution: An automated system employing intelligent techniques to analyse network traffic dynamically and detect phishing attempts in real time.

Project Objectives

01

Live Traffic Capture

Implement packet sniffing to capture DNS and HTTP network traffic in real time

02

Data Extraction

Parse network packets to extract domains, URLs, and relevant metadata for analysis

03

Rule-Based Detection

Apply heuristic techniques to identify suspicious patterns and known phishing indicators

04

ML Classification

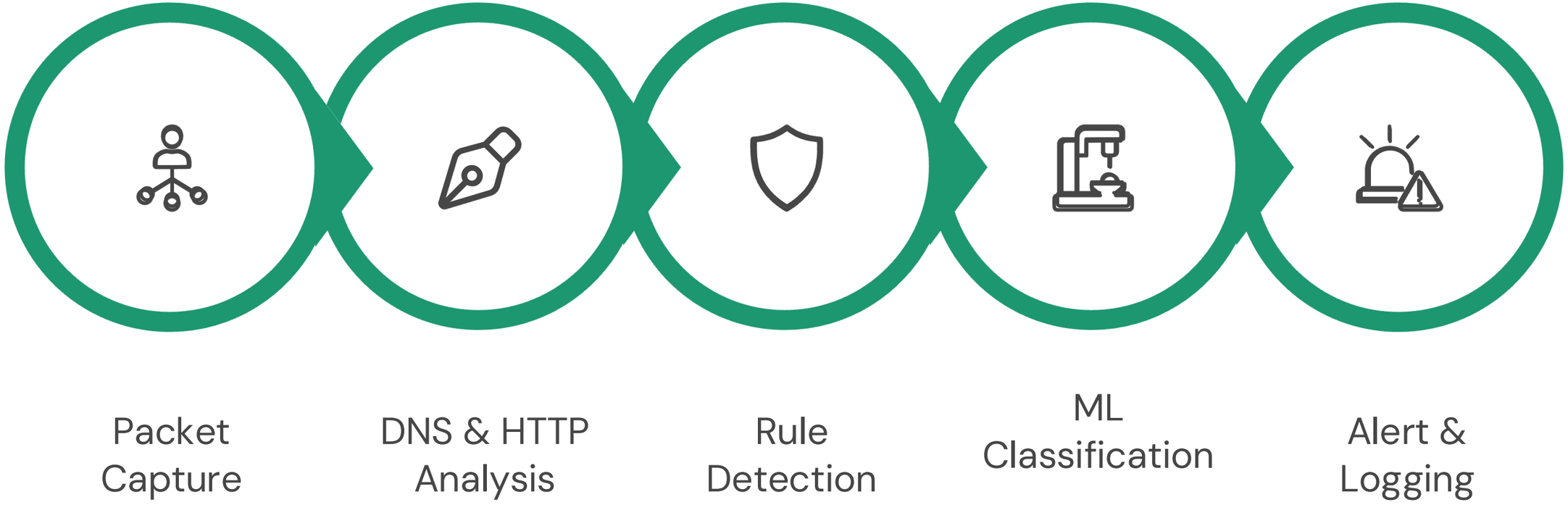
Leverage machine learning models to distinguish between phishing and legitimate URLs with improved accuracy

05

Alert System

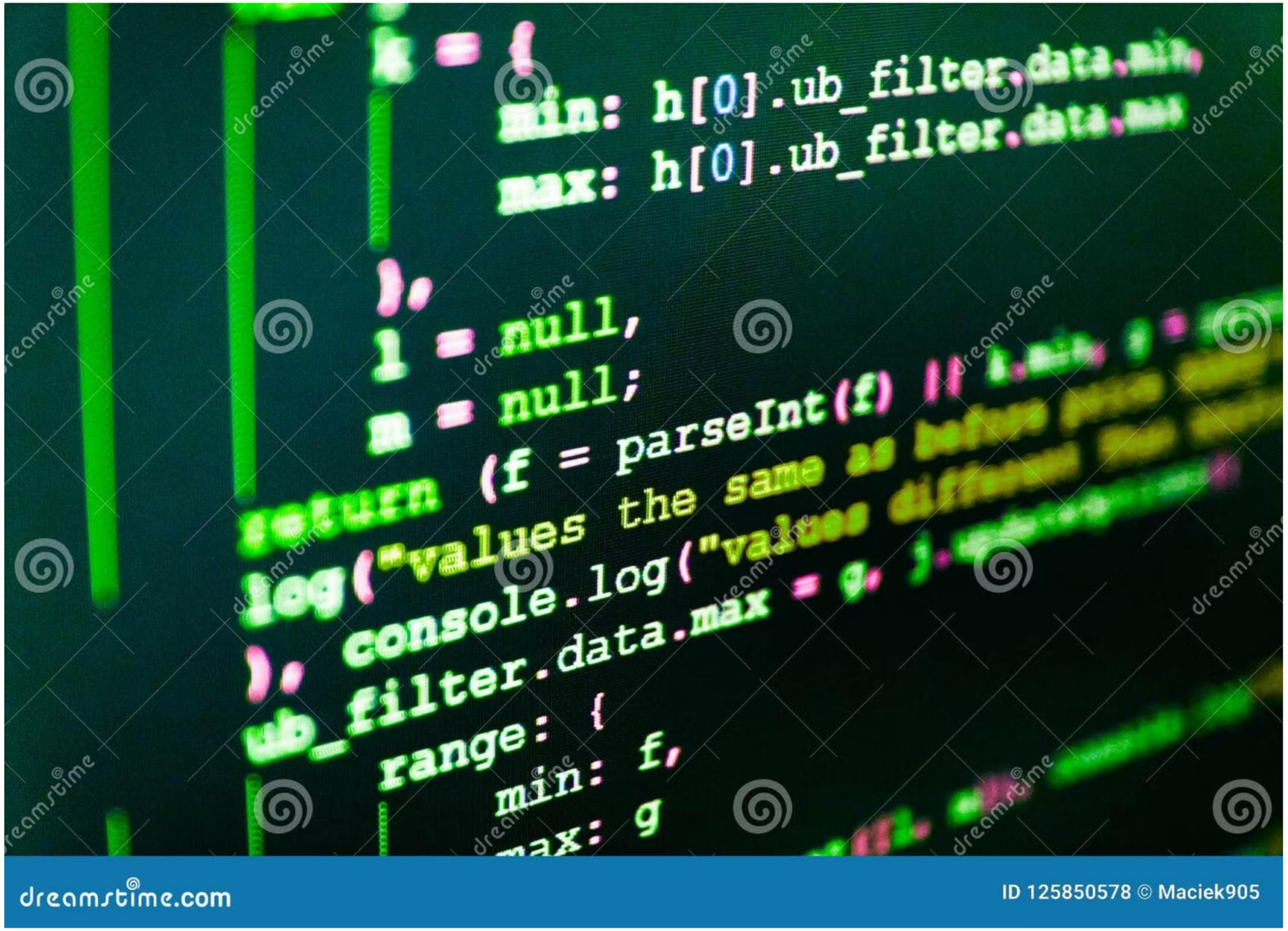
Generate real-time alerts and comprehensive logs for detected phishing activities

Solution Architecture & Methodology



Our solution employs a multi-layered approach combining packet sniffing, pattern recognition, and machine learning. Traffic flows through progressive analysis stages, with each layer adding intelligence to the detection process, culminating in actionable alerts and detailed logging for security teams.

Technical Stack



Core Technologies

Programming Language: Python 3.x

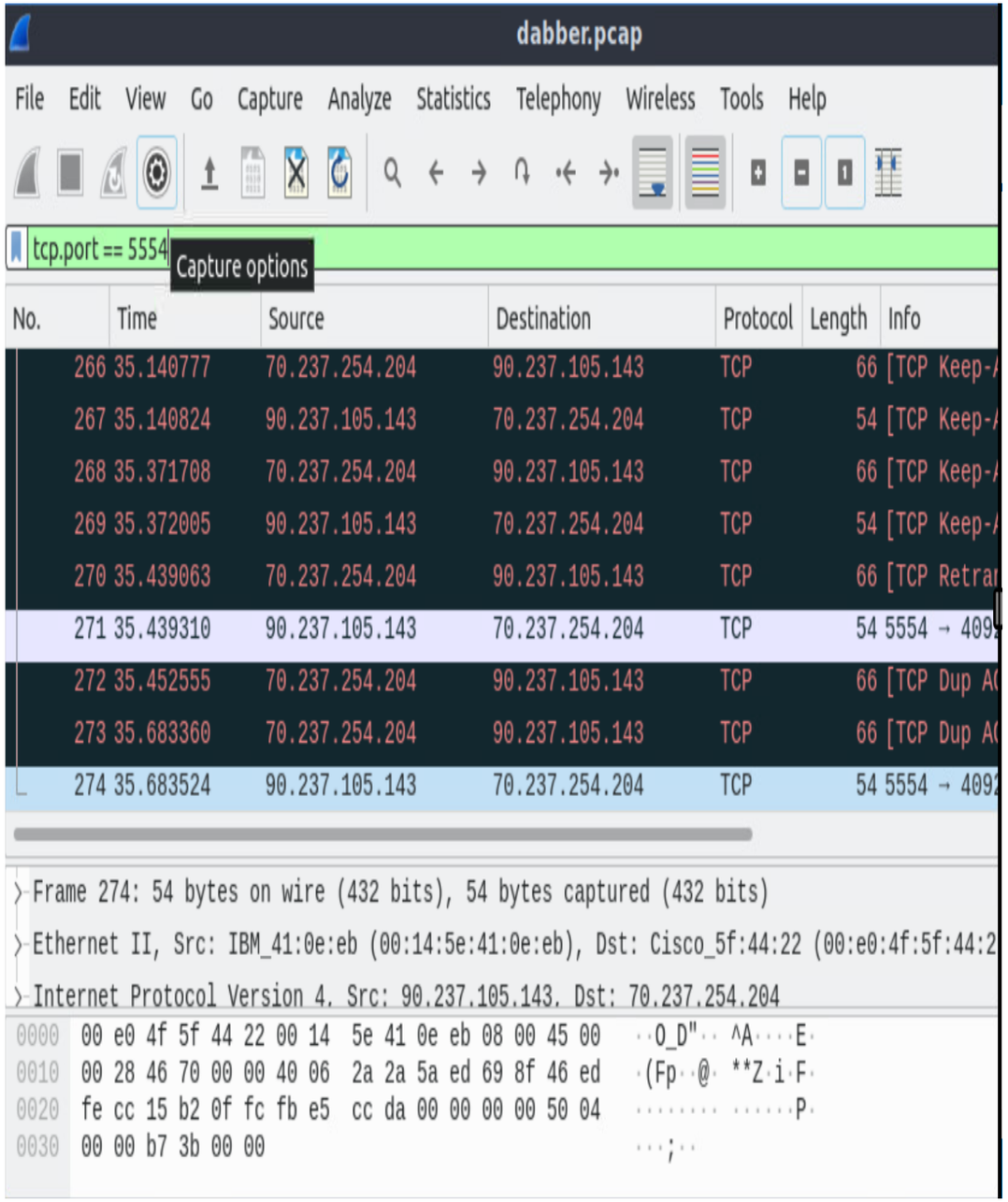
Key Libraries:

- **Scapy** – Packet capture and network analysis
- **Requests** – HTTP communication and URL validation
- **Scikit-learn** – Machine learning model training
- **Joblib** – Model serialisation and persistence

Development Environment: VS Code, GitHub
Operating Systems: Windows & Linux compatible

1.Overview:

The proposed system follows a hybrid approach combining rule-based detection and machine learning techniques to identify phishing attempts in real-time network traffic. The system continuously monitors DNS, HTTP, and HTTPS traffic and analyzes extracted data to detect suspicious activities.



2. Network Traffic Capture:

The system uses the Python library Scapy to capture live network packets. It listens to:

*DNS traffic (port 53)

*HTTP traffic (port 80)

*HTTPS traffic (TLS handshake)This allows the system to monitor user activity in real time.

3. Data Extraction:

a.DNS Analysis:

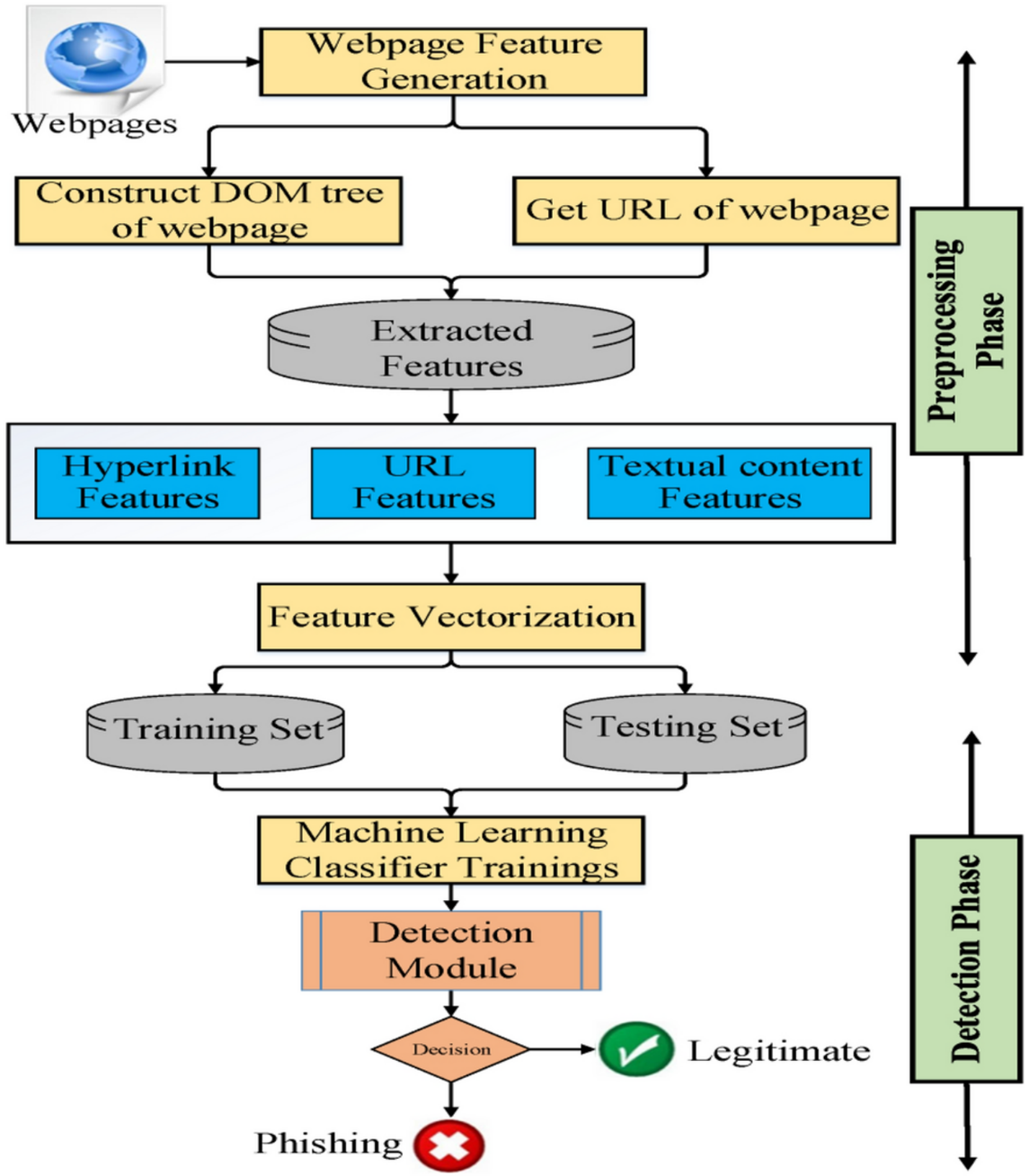
The system extracts domain names from DNS queries. These domains are analyzed to detect suspicious patterns.

b. HTTP Analysis:For HTTP traffic, the system extracts: Host and Path .These are combined to form a complete URL.

c. HTTPS Analysis (TLS SNI):For HTTPS traffic, the system extracts the Server Name Indication (SNI) from TLS packets, which reveals the domain name even in encrypted communication.

4. Feature Engineering:

Each URL/domain is converted into numerical features for machine learning. The features include:URL length,Number of dots (.), Number of hyphens (-), Presence of HTTPS, Presence of IP address, Number of digits,Number of special characters.These features help distinguish between legitimate and phishing URLs.



5. Machine Learning Model:

A ***Random Forest Classifier*** is used for phishing detection. The model is trained on a dataset of labeled URLs where:

* 0 represents legitimate websites.
*1 represents phishing websites .

The model outputs:

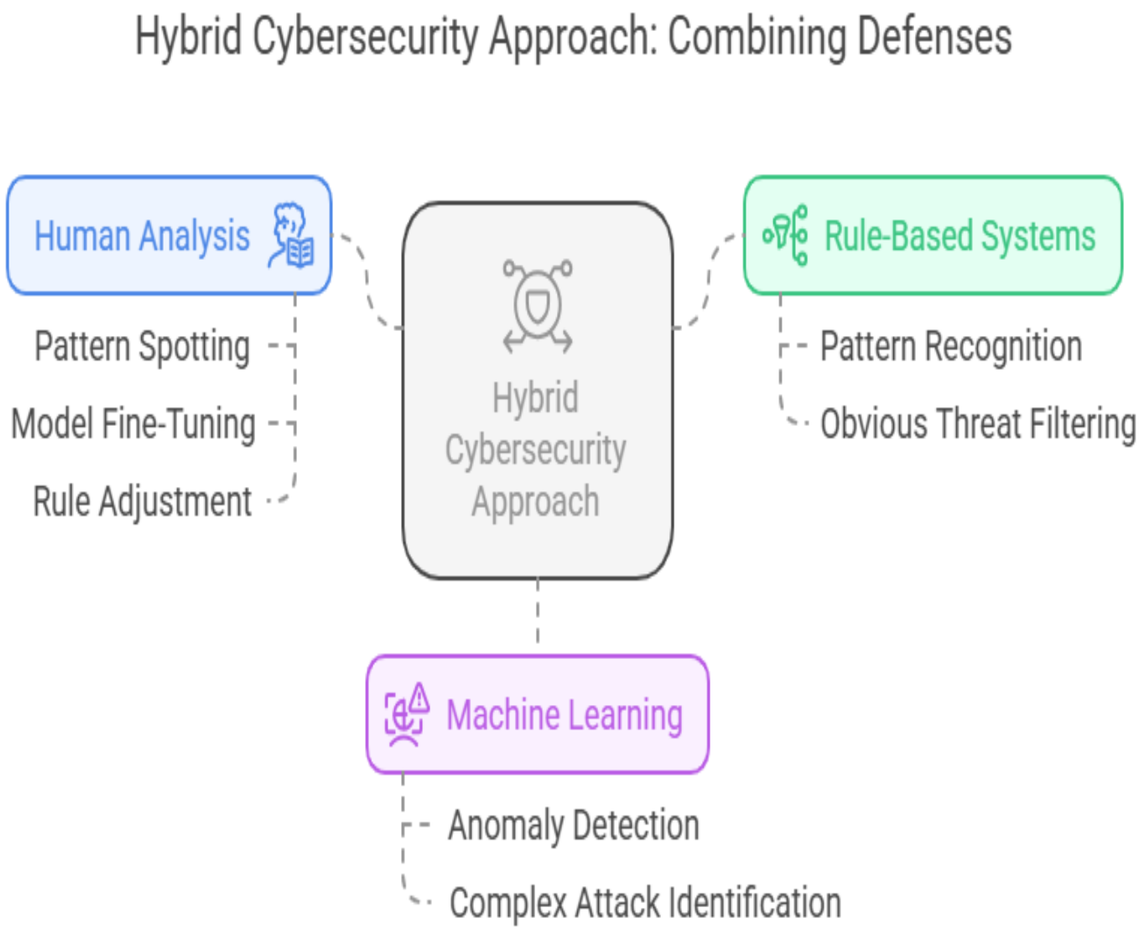
*Prediction (phishing or safe)
*Confidence score.

6. Rule-Based Detection:

In addition to ML, the system uses rule-based checks such as:

*Presence of suspicious keywords (e.g., login, verify, bank).
*Unusual URL length.
*IP address in URL .

This helps detect phishing attempts that may not be captured by the ML model.

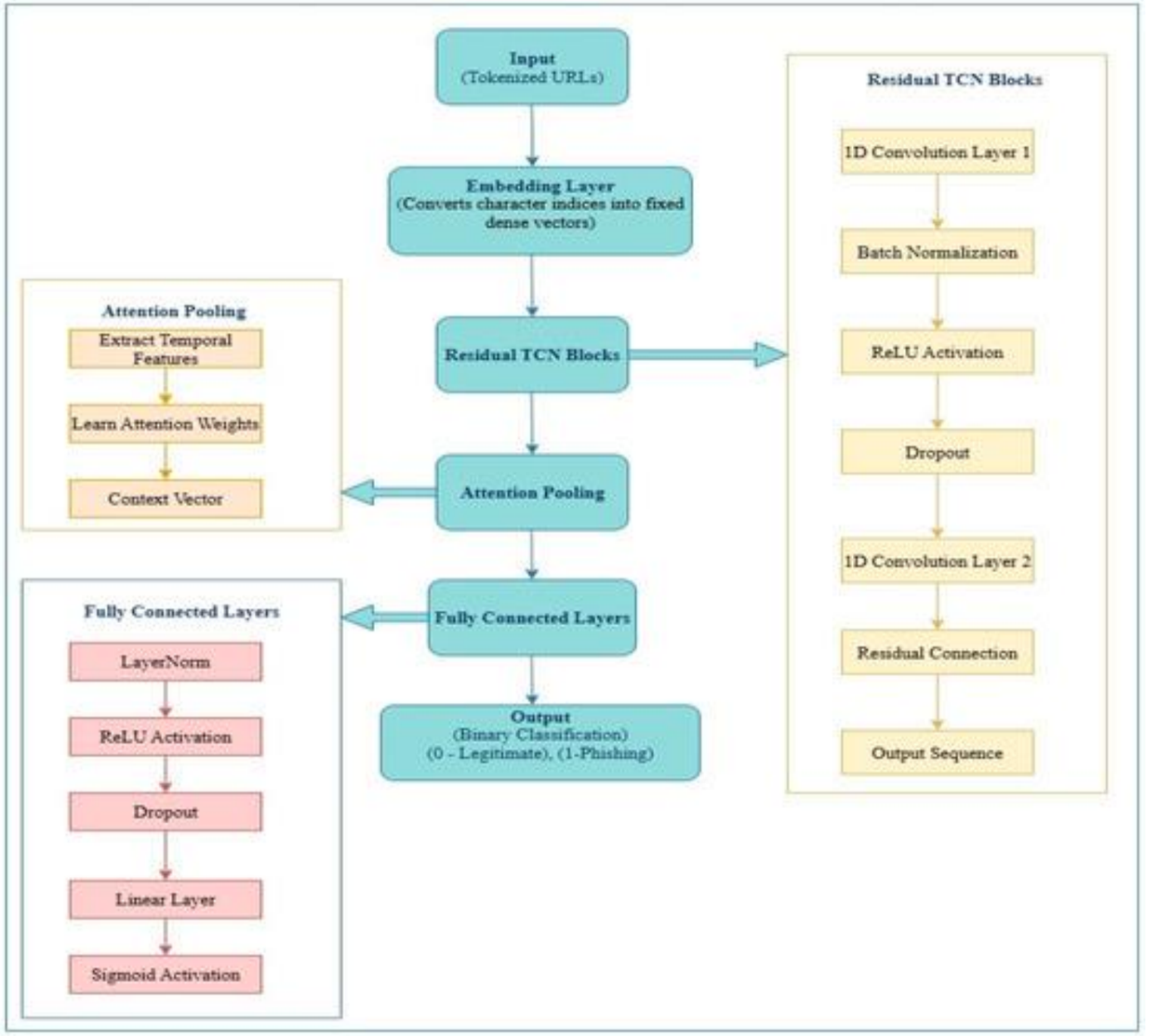


7. Hybrid Detection Mechanism:

The system combines both approaches:
*Rule-based detection.
*Machine learning prediction.
A URL is flagged as phishing if:
*It is suspicious and has moderate ML confidence, OR
*It has high ML confidence.
This reduces false positives and improves detection accuracy.

8. Logging and Storage:

Detected phishing attempts are:
*Logged into a text file
*Stored in a SQLite database
Each log contains:
*Timestamp.
*Source (DNS/HTTP/HTTPS).
*URL/domain.
*Confidence score.



9. Dashboard Visualization:

A Flask-based web dashboard is used to display:

*Real-time phishing logs.

*Confidence scores.

*Graphical analytics using Chart.js.

The dashboard updates automatically to show live network activity.

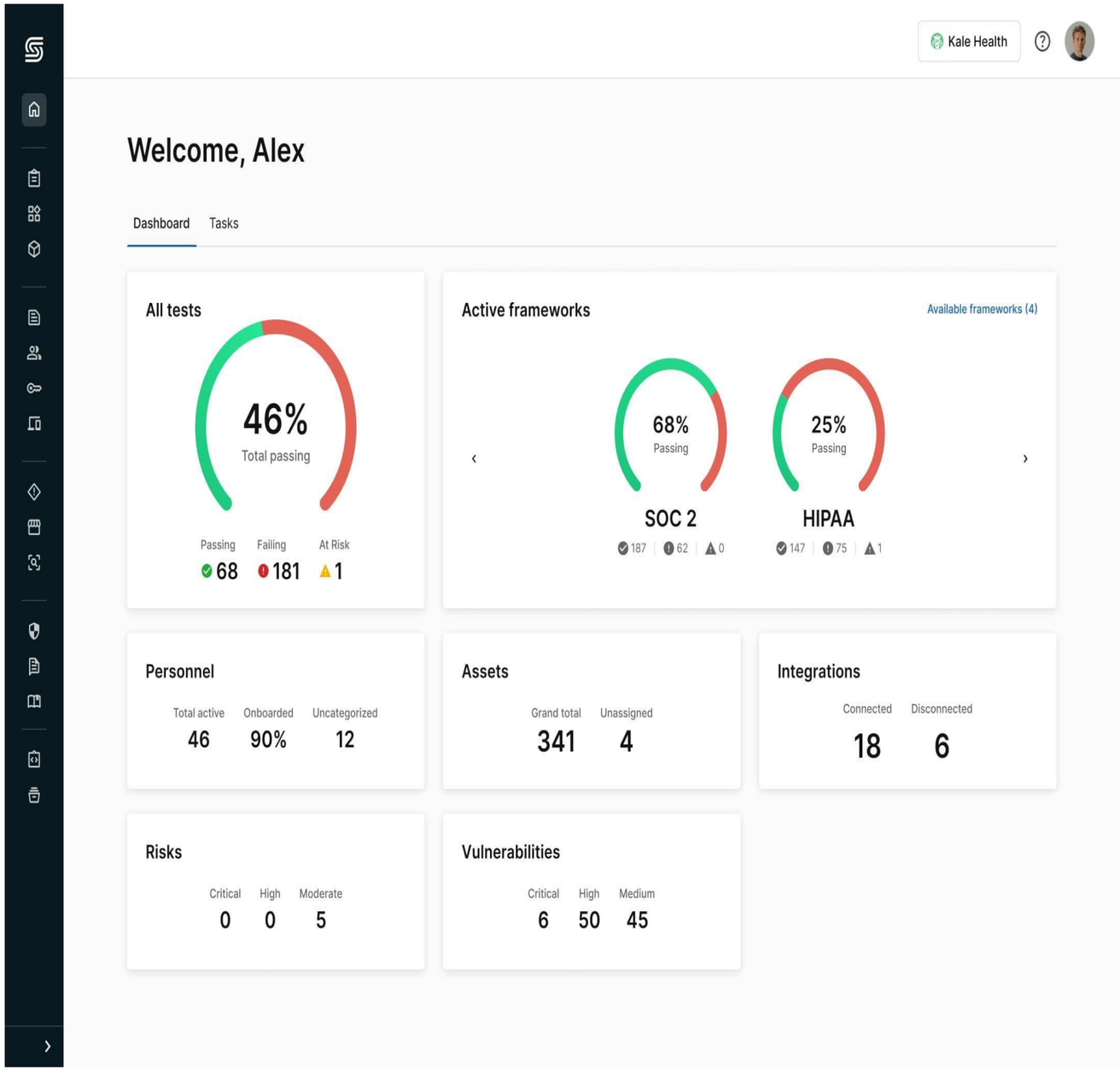
10. Noise Filtering and Optimization:

To improve performance:

*Duplicate URLs are filtered.

*Trusted domains (e.g., Microsoft, GitHub) are ignored.

*Clean formatting is applied to logs.



Summary:

The system provides a real-time, scalable, and efficient phishing detection solution by combining:

Network packet analysis.

Feature-based machine learning.

Rule-based heuristics.

Live visualization dashboard.

This approach ensures accurate detection while maintaining system performance.

System Requirements

Hardware Specifications

- **Processor:** Intel i3 or equivalent (higher recommended)
- **Memory:** Minimum 4 GB RAM
- **Storage:** 10 GB available space
- **Network:** Active network interface card

Software Requirements

- **OS:** Windows 10/11 or Linux distributions
- **Python:** Version 3.8 or higher
- **IDE:** Visual Studio Code
- **Version Control:** Git/GitHub
- **Libraries:** Scapy, Scikit-learn, Requests, Joblib

Project Outcomes & Future Roadmap

Expected Deliverables

1

Functional System

Real-time phishing detection with network traffic analysis

2

Improved Accuracy

ML-enhanced detection reducing false negatives

3

Reduced Dependency

Less reliance on static blacklists

4

Educational Value

Practical cybersecurity learning experience

Future Enhancements

- **Deep Learning:** Implement neural networks for enhanced pattern recognition
- **HTTPS Analysis:** Deep packet inspection for encrypted traffic
- **Web Dashboard:** Interactive monitoring interface with analytics
- **Browser Extension:** Client-side protection layer
- **Expanded Dataset:** Larger training corpus for improved accuracy

This project establishes a robust foundation for advanced cybersecurity solutions, combining rule-based logic with machine learning to protect users from evolving phishing threats.

Thank You